FIELD NOTES

Stripe Field Guide

Stripe fails quietly. A webhook endpoint stops delivering and the payments still succeed, so revenue looks normal while everything that should happen after a payment stops. Every script here is read only.

114 fixes, each one a written note with diagrams and a small Python and Node.js script that finds the problem and repairs it. All 114 have a script in the public repo.

Before you run anything. Every script starts in a dry run: it reports what it would change and writes nothing. Read that output first, then set `DRY_RUN=false` when you are satisfied it has understood your data.

Scripts: github.com/allanninal/stripe-fixes
Guides: allanninal.dev/stripe
All 14 repos: github.com/allanninal

The fixes

1 **3DS handoff breaks and requires_action intents pile up**

Card volume from Europe and India reads lower than your traffic says it should. Nothing fails. There are no declines to look at, no errors in the logs, and no support tickets, because from the customer's side the page simply did nothing. The intents stopped at requires_action and stayed there.

`stripe_requires_action.py`

`https://www.allanninal.dev/stripe/abandoned-requires-action-intents/`

`https://github.com/allanninal/stripe-fixes/tree/main/abandoned-requires-action-intents`

---

2 **account default API version is years behind the current one**

Nothing on a webhook endpoint explains this one. The pinned-endpoint note is about a value you can read straight off the endpoint object; this is the account-wide default sitting underneath it, and there is no API endpoint anywhere that returns it. It is why a parameter copied out of the current documentation comes back as no such parameter, and why renewal invoices Stripe generates for you arrive in a shape from two years ago.

`stripe_account_api_version.py`

`https://www.allanninal.dev/stripe/account-default-api-version-stale/`

`https://github.com/allanninal/stripe-fixes/tree/main/account-default-api-version-stale`

---

3 **the platform collects zero application fees on its charges**

The marketplace is working. Volume is up, sellers are getting paid, the Connect dashboard is busy. The platform's own revenue line reads zero, and has since launch. Nothing errored: every charge did exactly what it was told, which was to pass the whole amount through.

`stripe_application_fees.py`

https://www.allanninal.dev/stripe/application-fees-zero-on-platform/

https://github.com/allanninal/stripe-fixes/tree/main/application-fees-zero-on-platform

---

4 **automatic_tax is off on every invoice while selling abroad**

Stripe Tax was switched on eighteen months ago, the registrations were filed, and everyone moved on. The invoices going to Germany, France and the UK still carry no VAT, because enabling Tax in the Dashboard did nothing to the subscriptions the API had already created. Nothing errors. The totals are simply wrong, and the liability compounds every month.

`stripe_automatic_tax_off.py`

https://www.allanninal.dev/stripe/automatic-tax-disabled-everywhere/

https://github.com/allanninal/stripe-fixes/tree/main/automatic-tax-disabled-everywhere

---

5 **automatic_tax reports requires_location_inputs on live bills**

Stripe Tax is enabled, the integration passes automatic_tax[enabled]=true everywhere, and the totals look right on every invoice anyone has opened. They have been opening the wrong ones. A slice of the customers — the ones whose addresses were never captured properly — have been billed and paid with no tax calculated, and the invoices carrying that fact are already finalized and immutable.

`stripe_tax_location_status.py`

https://www.allanninal.dev/stripe/automatic-tax-requires-location-inputs/

https://github.com/allanninal/stripe-fixes/tree/main/automatic-tax-requires-location-inputs

---

6 **charges captured after AVS and CVC verification failed**

The dispute says the cardholder never made the purchase, and when you open the charge to build a case you find the billing postal code did not match the issuer's records and the security code was wrong. Stripe recorded both facts at the time. Nothing was configured to act on them, so the payment was captured and the goods shipped.

`stripe_avs_cvc_checks.py`

https://www.allanninal.dev/stripe/avs-cvc-fail-captured/

https://github.com/allanninal/stripe-fixes/tree/main/avs-cvc-fail-captured

---

7 **bank-debit intents stay in processing for over a week**

Bank-debit orders divide into two piles and neither is right. Some shipped the moment the customer clicked, and a few of those later bounced. The rest have never shipped at all, because nothing in the code ever looked at them again. The intents are all sitting in processing, which is where an ACH payment is supposed to sit — for a while.

`stripe_bank_debit_processing.py`

https://www.allanninal.dev/stripe/bank-debit-intents-stuck-processing/

https://github.com/allanninal/stripe-fixes/tree/main/bank-debit-intents-stuck-processing

---

8 **billing portal can't cancel, so customers charge back instead**

Your dispute rate is drifting up and the reasons cluster on one code: subscription_canceled. These are not fraudsters. They are customers who wanted to stop paying, could not find a way to do it, emailed support, waited, and then used the one cancellation mechanism that always works — their bank.

`stripe_portal_cancel_disabled.py`

https://www.allanninal.dev/stripe/billing-portal-cancel-disabled/

https://github.com/allanninal/stripe-fixes/tree/main/billing-portal-cancel-disabled

---

9 **no Billing Portal configuration, so portal sessions 400**

The Manage subscription button worked all through development. On the day it goes live it throws a 500, and the log underneath it reads No configuration provided and your default configuration has not been created. Nothing about the deploy changed the portal code, because the thing that is missing was never in the code.

`stripe_portal_configuration.py`

https://www.allanninal.dev/stripe/billing-portal-no-configuration/

https://github.com/allanninal/stripe-fixes/tree/main/billing-portal-no-configuration

---

10 **active subscriptions already committed to cancel at period end**

MRR is flat, the active-subscriber count is flat, and nothing in the billing data looks wrong. A month from now a fifth of it disappears in a week. The cancellations already happened; they just have not taken effect yet, and no report you have distinguishes a subscription that will renew from one that will not.

`stripe_pending_churn.py`

https://www.allanninal.dev/stripe/cancel-at-period-end-churn-backlog/

https://github.com/allanninal/stripe-fixes/tree/main/cancel-at-period-end-churn-backlog

---

11 **intents hardcode payment_method_types to card only**

You turned on iDEAL, Bancontact, Klarna and Link in the Dashboard weeks ago. The toggles are still on. The Payment Element in production still renders one card form, and conversion in the Netherlands is still flat. Nothing is broken and no error is thrown — the intent told Stripe exactly which methods to offer, and it named one.

`stripe_payment_method_coverage.py`

https://www.allanninal.dev/stripe/card-only-payment-method-types/

https://github.com/allanninal/stripe-fixes/tree/main/card-only-payment-method-types

---

12 **card_payments inactive disables transfers as well**

You read capabilities.transfers, saw inactive, fetched the capability, collected the two fields in its currently_due, and submitted them. Stripe accepted the update. The capability is still inactive, its currently_due is now empty, and transfers still fail. Nothing you can see on that capability explains it, because the requirement holding it down is filed under a different one.

`stripe_capability_coupling.py`

https://www.allanninal.dev/stripe/card-payments-inactive-cascades/

https://github.com/allanninal/stripe-fixes/tree/main/card-payments-inactive-cascades

---

13 **saved cards expire within 60 days and nothing warns anyone**

The churn chart has lumps in it, and the lumps are always at the end of a month. Nobody cancelled. Their card expired, the renewal declined, and the first the customer heard about it was a failed-payment email that reads like an accusation. The expiry date was sitting in the API from the day they saved the card.

`stripe_card_expiry_window.py`

https://www.allanninal.dev/stripe/cards-expiring-within-60-days/

https://github.com/allanninal/stripe-fixes/tree/main/cards-expiring-within-60-days

---

14 **the endpoint listens for charge.succeeded, not payment_intent**

This does not present as a webhook problem, because the webhook arrives. It presents as a data problem: the object in the payload has empty metadata, no customer, and nothing that maps the payment back to a cart or a user. The metadata is on the PaymentIntent. The endpoint is subscribed to the Charge.

`stripe_charge_event_drift.py`

https://www.allanninal.dev/stripe/charge-events-but-paymentintent-integration/

https://github.com/allanninal/stripe-fixes/tree/main/charge-events-but-paymentintent-integration

---

15 **session status is complete but payment_status is still unpaid**

The handler listens for checkout.session.completed, marks the order paid and ships it. Most of the time that is correct. For ACH, SEPA and every other delayed payment method it is a guess, and a few days later some of those guesses come back as checkout.session.async_payment_failed with the goods already gone.

`stripe_unpaid_complete_sessions.py`

https://www.allanninal.dev/stripe/checkout-complete-payment-unpaid/

https://github.com/allanninal/stripe-fixes/tree/main/checkout-complete-payment-unpaid

---

16 **embedded Checkout never redirects and return_url is null**

The form is embedded in your own page, which is the whole point of it. Then a customer pays with iDEAL: they are sent to their bank, they authenticate, and they come back to nowhere, because return_url is null and the return leg has no destination. To them the payment failed. To Stripe it did not.

`stripe_checkout_return_urls.py`

https://www.allanninal.dev/stripe/checkout-embedded-no-return-url/

https://github.com/allanninal/stripe-fixes/tree/main/checkout-embedded-no-return-url

---

17 **most Checkout Sessions expire unpaid and nobody is told**

Sessions are being created at a healthy rate and the revenue line is flat. There is no error, no failed payment, no declined card — the sessions simply stop existing. Stripe does emit an event when one lapses, exactly 24 hours after it was created, and almost nobody is subscribed to it.

`stripe_checkout_abandonment.py`

https://www.allanninal.dev/stripe/checkout-expired-session-share/

https://github.com/allanninal/stripe-fixes/tree/main/checkout-expired-session-share

---

18 **guest checkouts finish with customer null and can't be linked**

Somebody writes in asking for a copy of a receipt from March. You search the Dashboard for their email address and find four separate payments, no Customer record, no purchase history, and nothing you can open the Billing Portal against. Every one of those four checkouts completed perfectly. None of them left behind anything that ties the four together.

`stripe_checkout_guests.py`

https://www.allanninal.dev/stripe/checkout-guest-customer-null/

https://github.com/allanninal/stripe-fixes/tree/main/checkout-guest-customer-null

---

19 **expired Checkout Sessions are never recovered by email**

Somebody asks why there is no abandoned-cart email. The answer is not that nobody built one. It is that there is nothing to send: Checkout will mint a recovery link for every session that lapses, but only for sessions created with recovery switched on, and switching it on afterwards does nothing at all.

`stripe_checkout_recovery.py`

https://www.allanninal.dev/stripe/checkout-recovery-never-enabled/

https://github.com/allanninal/stripe-fixes/tree/main/checkout-recovery-never-enabled

---

20 **Checkout Sessions carry no ID that maps back to your order**

Support is reconciling payments by matching an email address and an amount against the order table by hand. It works, mostly, until two people buy the same thing on the same day. Then a dispute arrives on one of those charges and nobody can say with confidence which order it was, which is a bad position to answer a dispute from.

`stripe_checkout_reconciliation.py`

https://www.allanninal.dev/stripe/checkout-sessions-unreconcilable/

https://github.com/allanninal/stripe-fixes/tree/main/checkout-sessions-unreconcilable

---

21 **a Connect platform has no endpoint for connected accounts**

The endpoint is enabled. It is subscribed to account.updated. It has been in the dashboard for two years and it delivers thousands of events a week without a single failure. And it has never once received an event about a connected account, because the thing that decides whether it does is not in enabled_events and is not returned on the object you are looking at.

`stripe_connect_webhook_scope.py`

https://www.allanninal.dev/stripe/connect-platform-missing-account-updated/

https://github.com/allanninal/stripe-fixes/tree/main/connect-platform-missing-account-updated

---

22 **connect_reserved grows as connected accounts go negative**

Payouts to the platform's own bank are smaller than the ledger says they should be, and the shortfall grows a little every month. No payout failed, no charge was refunded twice, and the Dashboard's headline balance looks plausible. The money is not missing — it is reserved, against connected accounts whose own balances have gone below zero.

`stripe_connect_reserve.py`

https://www.allanninal.dev/stripe/connect-reserved-balance-growing/

https://github.com/allanninal/stripe-fixes/tree/main/connect-reserved-balance-growing

---

23 **a connected account sits with charges_enabled false**

A seller emails support to say their checkout has been broken for two weeks. Nobody on the platform side saw anything: no alert, no failed job, no error in the logs. The platform's own Stripe account is healthy, payments are flowing, the graphs are flat and normal. The account that stopped working is one of four hundred, and the only field that would have told you is one nobody was reading.

`stripe_connect_charges_disabled.py`

https://www.allanninal.dev/stripe/connected-accounts-charges-disabled/

https://github.com/allanninal/stripe-fixes/tree/main/connected-accounts-charges-disabled

---

24 **current_deadline passes before anyone collects the fields**

Nine sellers lost payouts on the same Monday. All nine had been processing happily for months, all nine had payouts_enabled: true on Sunday night, and all nine had the same Unix timestamp sitting in requirements.current_deadline since the middle of the previous month. Nothing read it. It is a number, not a flag, and every check anyone had written asked yes-or-no questions.

`stripe_current_deadline.py`

https://www.allanninal.dev/stripe/current-deadline-passes-unwatched/

https://github.com/allanninal/stripe-fixes/tree/main/current-deadline-passes-unwatched

---

25 **customers have no address, so tax and SCA exemptions fail**

An invoice will not finalize. The error is customer_tax_location_invalid, which sounds like a Stripe Tax configuration problem and is not: the registrations are fine, the rates are fine, and the customer simply has no address on file. Your own database has their shipping address. Stripe's copy of the Customer has address: null, and it has been that way since the integration was written.

`stripe_customer_address.py`

https://www.allanninal.dev/stripe/customers-missing-address/

https://github.com/allanninal/stripe-fixes/tree/main/customers-missing-address

---

26 **customers have no email, so Stripe sends no receipts**

A dispute arrives with the reason “unrecognised”. The cardholder is a real, current customer, and they are not lying: they never got a receipt, so the only thing linking that line on their statement to your business is a descriptor they have never seen before. The Customer record has an email column and it is empty.

`stripe_customers_missing_email.py`

https://www.allanninal.dev/stripe/customers-missing-email/

https://github.com/allanninal/stripe-fixes/tree/main/customers-missing-email

---

27 **enabled_events lists event types that are dead or rejected**

Card-expiry reminder emails stopped going out. Nobody can say when, because nothing failed: the handler branch simply never runs, and a branch that never runs produces no logs. Separately, and apparently unrelatedly, an attempt to add one new event type to the endpoint comes back with You do not have access to the event types, naming a type that has been in the list for years.

`stripe_dead_event_types.py`

https://www.allanninal.dev/stripe/dead-or-rejected-enabled-events/

https://github.com/allanninal/stripe-fixes/tree/main/dead-or-rejected-enabled-events

---

28 **disputes are hours from due_by with no evidence attached**

A customer disputed a charge three weeks ago. The notification went to the shared billing inbox, where it sat under invoices. The dispute closed yesterday as lost, the funds went back, the dispute fee did not come back, and the delivery confirmation that would have answered it was in a support ticket the whole time.

`stripe_dispute_deadlines.py`

https://www.allanninal.dev/stripe/dispute-deadline-72h-no-evidence/

https://github.com/allanninal/stripe-fixes/tree/main/dispute-deadline-72h-no-evidence

---

29 **dispute activity is above the 0.75% excessive threshold**

Nobody in the company can say what the dispute rate is. Individually the disputes look ordinary and each one gets handled on its own terms, so the trend never gets discussed. The first hard signal is an email from Stripe naming a card network monitoring programme, a fine, and a remediation plan with a deadline on it.

`stripe_dispute_rate.py`

https://www.allanninal.dev/stripe/dispute-rate-above-threshold/

https://github.com/allanninal/stripe-fixes/tree/main/dispute-rate-above-threshold

---

30 **disputes closed as lost were never actually contested**

Somebody finally asks what the dispute win rate is. The Dashboard says most of them are lost, and the conclusion in the room is that disputes are unwinnable and not worth the effort. Nobody in the room can say how many of those losses were ever answered, and until someone can, that conclusion is unsupported.

`stripe_dispute_forfeits.py`

https://www.allanninal.dev/stripe/disputes-lost-without-response/

https://github.com/allanninal/stripe-fixes/tree/main/disputes-lost-without-response

---

31 **draft invoices blocked on customer_tax_location_invalid**

A handful of customers have not been billed since Stripe Tax was switched on, and their subscriptions all read active. Their renewal invoices were created on schedule, priced correctly, and then refused at the last step: Stripe Tax cannot work out where the customer is, so it will not finalize the invoice, and an unfinalized invoice is never sent and never charged.

`stripe_tax_blocked_drafts.py`

https://www.allanninal.dev/stripe/draft-invoices-blocked-by-tax-location/

https://github.com/allanninal/stripe-fixes/tree/main/draft-invoices-blocked-by-tax-location

---

32 **draft invoices sit for months and never finalize**

Somebody asks why the March figure is lower than the March that was forecast, and nobody can answer it from the Dashboard, because the money is not missing and it is not late. It is sitting in invoices that were built, itemised, priced correctly, and then never sent: no number, no PDF, no hosted page, no email. Stripe cannot collect on an invoice that has not been finalized, and these have been drafts since March.

`stripe_draft_invoices.py`

https://www.allanninal.dev/stripe/draft-invoices-never-finalized/

https://github.com/allanninal/stripe-fixes/tree/main/draft-invoices-never-finalized

---

33 **dunning ran out of retries and no attempt is scheduled**

A card expired in May. Stripe retried it eight times over two weeks, each attempt failed, and then it stopped, which is exactly what it is supposed to do. Nothing announced the end of that sequence. The invoice is still open, the customer still has access, and the last human to look at the account was the one who set up the subscription.

`stripe_dunning_exhausted.py`

https://www.allanninal.dev/stripe/dunning-retries-exhausted/

https://github.com/allanninal/stripe-fixes/tree/main/dunning-retries-exhausted

---

34 **duplicate customers share an email and split billing**

A customer writes in to say they were charged twice this month. Support finds their subscription, checks it, and it billed once. The second charge is on a different Customer record with the same email address, created the day they resubscribed, and it has its own card, its own subscription and its own renewal date.

`stripe_duplicate_customers.py`

https://www.allanninal.dev/stripe/duplicate-customers-same-email/

https://github.com/allanninal/stripe-fixes/tree/main/duplicate-customers-same-email

---

35 **two endpoints share one URL, so every event is handled twice**

Two order rows for one payment. Two fulfilment emails. A customer credited twice. The handler reads correctly, it passes its tests, and it does not misbehave locally — because locally there is one endpoint, and in production there are two pointing at the same URL.

`stripe_duplicate_endpoints.py`

https://www.allanninal.dev/stripe/duplicate-endpoints-same-url/

https://github.com/allanninal/stripe-fixes/tree/main/duplicate-endpoints-same-url

---

36 **actionable early fraud warnings were never refunded**

A batch of fraud disputes lands in the same week, all on charges from a month earlier. Each of those charges already carried an early fraud warning at the time, sitting in the API with actionable set to true, which is Stripe saying in as many words that you could still refund it. Nobody was subscribed to the event and nobody swept for it, so the window opened and closed unobserved.

`stripe_efw_actionable.py`

https://www.allanninal.dev/stripe/efw-actionable-not-refunded/

https://github.com/allanninal/stripe-fixes/tree/main/efw-actionable-not-refunded

---

37 **elevated-risk charges captured with no manual review**

Disputes keep arriving for payments that went through weeks ago, and the chargeback rate is drifting toward the number the card networks care about. Open any one of them and Stripe already knew: the charge is marked elevated risk. Nobody was ever shown it, because nothing on the account was configured to show anybody anything.

`stripe_elevated_risk_review.py`

https://www.allanninal.dev/stripe/elevated-risk-charges-no-review/

https://github.com/allanninal/stripe-fixes/tree/main/elevated-risk-charges-no-review

---

38 **endpoints render events at different pinned API versions**

This is not one endpoint pinned to an ancient version, where every consumer at least agrees on the shape. This is two endpoints that disagree with each other. The same logical event goes to both, one service reads invoice.subscription and the other reads invoice.parent, and only one of them falls over. Both are configured exactly as somebody intended, six months apart.

`stripe_endpoint_version_drift.py`

https://www.allanninal.dev/stripe/endpoint-api-version-drift/

https://github.com/allanninal/stripe-fixes/tree/main/endpoint-api-version-drift

---

39 **a webhook endpoint is pinned to an ancient api_version**

The signature verifies. The event arrives. And then the object inside it is missing half the fields the current SDK expects, so the deserializer hands back an empty Optional or a null cast, and nobody throws anything. Fetching the same object straight from the API returns it complete, which makes the whole thing look like Stripe sending empty payloads.

`stripe_endpoint_api_version.py`

https://www.allanninal.dev/stripe/endpoint-api-version-pinned-stale/

https://github.com/allanninal/stripe-fixes/tree/main/endpoint-api-version-pinned-stale

---

40 **events still show pending_webhooks hours after they fired**

Most payments are processed and some are not. There is no pattern anyone can see: the same customer, the same product, the same code path, and one order exists while the next does not. The endpoint is enabled, the Dashboard shows no banner, and your logs show the handler running successfully for every request it received.

`stripe_pending_webhooks.py`

https://www.allanninal.dev/stripe/events-with-pending-webhooks/

https://github.com/allanninal/stripe-fixes/tree/main/events-with-pending-webhooks

---

41 **manual-capture holds expire before anyone captures them**

A customer messages to say the pending charge disappeared from their statement, and they are pleased about it. You go to capture the payment, and Stripe answers charge_expired_for_capture. The order shipped. The authorization is gone, the money was never taken, and no retry exists that can take it now.

`stripe_manual_capture_holds.py`

https://www.allanninal.dev/stripe/expired-manual-capture-holds/

https://github.com/allanninal/stripe-fixes/tree/main/expired-manual-capture-holds

---

42 **saved cards are already expired but still attached**

MRR leaks by a percent or two every month and the churn report calls it voluntary. It is not: these customers never chose to leave. Their saved card expired, the renewal failed with expired_card, the dunning emails went to an inbox they do not read, and the account settings page still shows the card as though it works.

`stripe_expired_cards.py`

https://www.allanninal.dev/stripe/expired-saved-cards-attached/

https://github.com/allanninal/stripe-fixes/tree/main/expired-saved-cards-attached

---

43 **no external account can settle the account's currency**

The payout call comes back with Sorry, you don't have any external accounts in that currency (usd). The account plainly has a bank account: you can see it in the Dashboard, the seller added it during onboarding, and it has been sitting there for months. It is an Australian bank account, the balance is in dollars, and there is no arrangement under which one settles the other.

`stripe_settlement_currency.py`

https://www.allanninal.dev/stripe/external-account-currency-mismatch/

https://github.com/allanninal/stripe-fixes/tree/main/external-account-currency-mismatch

---

44 **a bank account sits at status errored and payouts stop**

A seller's payout failed in July. Somebody looked, saw one failure, assumed a temporary bank problem and moved on. It is now September, the seller's balance has grown to five figures, and there have been no further failed payouts at all — which is exactly what everyone took as evidence that the problem had resolved itself.

`stripe_external_account_errored.py`

https://www.allanninal.dev/stripe/external-account-errored/

https://github.com/allanninal/stripe-fixes/tree/main/external-account-errored

---

45 **future_requirements will revoke a capability on a date**

Fourteen connected accounts stopped taking payments on the same Thursday morning. All of them were fully verified. None of them had anything in requirements the night before, and the monitor that reads requirements every hour never made a sound. The fields that disabled them had been sitting on the same objects for six weeks, in a different hash, with the date printed on it.

`stripe_future_requirements.py`

https://www.allanninal.dev/stripe/future-requirements-deadline-ignored/

https://github.com/allanninal/stripe-fixes/tree/main/future-requirements-deadline-ignored

---

46 **highest-risk charges succeed instead of being blocked**

The Radar page says the highest-risk block rule is on. It has been on since the account was created, nobody has touched it, and it is doing nothing. Somewhere below it is an allow rule that somebody added to stop blocking a partner's traffic, and an allow rule wins against everything, including Stripe's own defaults.

`stripe_highest_risk_succeeded.py`

https://www.allanninal.dev/stripe/highest-risk-charges-succeeded/

https://github.com/allanninal/stripe-fixes/tree/main/highest-risk-charges-succeeded

---

47 **reused idempotency keys hit 409 idempotency_key_in_use**

The keys are being sent. That is what makes this different from payment requests with no key at all, where the header is simply absent. Here every request carries one, and the same one keeps turning up on requests that are not the same operation — so under load a slice of checkouts fails with 409 idempotency_key_in_use, and a day later the protection stops working entirely and the duplicates come back.

`stripe_idempotency_key_reuse.py`

https://www.allanninal.dev/stripe/idempotency-key-reuse-conflict/

https://github.com/allanninal/stripe-fixes/tree/main/idempotency-key-reuse-conflict

---

48 **incomplete_expired volume means confirmation is broken**

Sign-ups are steady, marketing spend is steady, and paid subscriptions are down. There are no failed payments to look at, because no payment was ever attempted. Filter the subscription list to incomplete_expired and there they are: a few hundred people who thought they had subscribed, in a state that Stripe considers finished.

`stripe_incomplete_expired_rate.py`

https://www.allanninal.dev/stripe/incomplete-expired-signup-leak/

https://github.com/allanninal/stripe-fixes/tree/main/incomplete-expired-signup-leak

---

49 **inquiries sit unanswered and escalate into chargebacks**

Chargebacks appear to arrive from nowhere. Somebody pulls the history for one of them and finds it was visible in the API eleven days earlier, as an inquiry, with a deadline and a place to put evidence. Nothing was wrong with the alerting: the sweep looked for disputes whose status was needs_response, and at that point the status was warning_needs_response.

`stripe_dispute_inquiries.py`

https://www.allanninal.dev/stripe/inquiry-needs-response-ignored/

https://github.com/allanninal/stripe-fixes/tree/main/inquiry-needs-response-ignored

---

50 **cardholder requirements.past_due keeps every card inactive**

The card was issued in the morning, handed to an employee at lunch, and declined at a card reader by two. The dashboard shows the card as inactive. Activating it does nothing. Nothing about the card object explains why, because the reason is not on the card — it is a list of two missing strings on the cardholder the card belongs to, one of which is an IP address.

`stripe_issuing_cardholder_requirements.py`

https://www.allanninal.dev/stripe/issuing-cardholder-requirements-past-due/

https://github.com/allanninal/stripe-fixes/tree/main/issuing-cardholder-requirements-past-due

---

51 **legacy card sources still live under customer.sources**

Half your customers renew without incident. The other half fail with Cannot charge a customer that has no active card, and when you open one of them in the Dashboard there is a card sitting right there. Both statements are true. The card is in a store the code that renews them cannot see.

`stripe_legacy_card_sources.py`

https://www.allanninal.dev/stripe/legacy-card-sources-still-attached/

https://github.com/allanninal/stripe-fixes/tree/main/legacy-card-sources-still-attached

---

52 **charges have a null payment_intent, which means the legacy Charges API**

European card volume declines at two or three times the rate of everything else, and nobody can find a cause. The cards are good. Radar is not blocking them. And 3D Secure never appears — not on a single one of these payments, ever, on any card from any issuer. That last detail is the whole diagnosis.

`stripe_legacy_charges.py`

https://www.allanninal.dev/stripe/legacy-charges-api-no-payment-intent/

https://github.com/allanninal/stripe-fixes/tree/main/legacy-charges-api-no-payment-intent

---

53 **metered subscription items with no usage reported**

The invoice goes out with the usage line at zero. The customer has been hammering the API all month and your own dashboards say so, but Stripe has no record of a single unit. The subscription is active, the price is metered, nothing has errored anywhere, and the invoice has already finalized, which is the point past which usage cannot be added to it.

`stripe_metered_usage.py`

https://www.allanninal.dev/stripe/metered-items-with-no-usage-reported/

https://github.com/allanninal/stripe-fixes/tree/main/metered-items-with-no-usage-reported

---

54 **EU business invoices with no VAT number miss reverse charge**

A German customer's finance team refuses the invoice. It has their company name on it, it has VAT added to it, and it has no VAT number and no reverse-charge notice anywhere on the document — so as far as their accounts payable is concerned it is a consumer receipt, and they cannot reclaim anything against it. Stripe did nothing wrong: with no tax ID on the customer, a business buyer is a consumer.

`stripe_eu_vat_ids.py`

https://www.allanninal.dev/stripe/missing-customer-tax-ids-b2b-eu/

https://github.com/allanninal/stripe-fixes/tree/main/missing-customer-tax-ids-b2b-eu

---

55 **nothing subscribes to disputes or early fraud warnings**

The first anyone knows about a chargeback is an email from Stripe about a deadline, or an unexplained dip in the balance. By the time somebody opens the dashboard, several days of the evidence window are gone. Separately and more expensively, the order that caused it shipped a week ago, even though the issuer had already flagged the card.

`stripe_dispute_events.py`

https://www.allanninal.dev/stripe/missing-dispute-and-fraud-events/

https://github.com/allanninal/stripe-fixes/tree/main/missing-dispute-and-fraud-events

---

56 **payment-creating requests carry no idempotency key**

A customer is charged twice. It happens perhaps once a week, always to someone on a phone, always during a moment when the network was bad, and never once on a developer machine. There is no bug in the checkout code. There is a missing HTTP header, and the events already recorded which requests were sent without it.

`stripe_idempotency_keys.py`

https://www.allanninal.dev/stripe/missing-idempotency-keys-on-payments/

https://github.com/allanninal/stripe-fixes/tree/main/missing-idempotency-keys-on-payments

---

57 **no endpoint subscribes to any payment failure event**

Every payment that succeeds is handled. Every payment that fails is nothing at all: no event, no branch, no row, no email. The cart stays in processing until somebody writes in, and on the billing side a declined renewal starts a dunning sequence that the application knows nothing about while it is happening.

`stripe_payment_failure_events.py`

https://www.allanninal.dev/stripe/missing-payment-failure-events/

https://github.com/allanninal/stripe-fixes/tree/main/missing-payment-failure-events

---

58 **payout.failed is unsubscribed so failures go unseen for days**

Money stopped arriving in the bank account. Nothing alerted, nothing errored, and the balance in Stripe kept climbing. On a Connect platform it is worse: the sellers notice before the platform does, and they notice by not being paid.

`stripe_payout_events.py`

https://www.allanninal.dev/stripe/missing-payout-failed/

https://github.com/allanninal/stripe-fixes/tree/main/missing-payout-failed

---

59 **no statement descriptor, so customers dispute what they see**

Every month a handful of disputes arrive with the reason unrecognized or duplicate, from customers who bought the thing, kept the thing, and are perfectly happy with it. They looked at a bank statement, saw a line that meant nothing to them, and did the sensible thing. You pay the dispute fee for each one.

`stripe_statement_descriptor.py`

https://www.allanninal.dev/stripe/missing-statement-descriptor/

https://github.com/allanninal/stripe-fixes/tree/main/missing-statement-descriptor

---

60 **customer.subscription.deleted is missing, so access never ends**

A support ticket arrives from somebody trying to be helpful: they cancelled two months ago and can still log in and use everything. You check Stripe and they really did cancel, on time, and the subscription really is canceled. The revenue reporting is correct. It is only the entitlement in your own database that has never been told.

`stripe_subscription_deleted_events.py`

https://www.allanninal.dev/stripe/missing-subscription-deleted/

https://github.com/allanninal/stripe-fixes/tree/main/missing-subscription-deleted

---

61 **recent events carry two different api_version values**

Nothing here is a configuration field you can go and correct. The drift note is about endpoints disagreeing; this is about the stored events themselves. Event objects are immutable and rendered at whatever the account default was when they occurred, so an upgrade cuts a hard line across the 30-day window. Everything after it is one shape, everything before it is another, and a backfill walks straight through the boundary.

`stripe_event_version_boundary.py`

https://www.allanninal.dev/stripe/mixed-event-api-versions/

https://github.com/allanninal/stripe-fixes/tree/main/mixed-event-api-versions

---

62 **elevated-risk card charges are captured with no 3DS**

Fraud disputes on card-not-present payments are being lost one after another, and each response comes back the same way. Somebody checks whether the liability shift applies and finds that none of the disputed charges went through 3D Secure at all: payment_method_details.card.three_d_secure is null on every one, including the ones Radar had already scored as elevated risk.

`stripe_3ds_coverage.py`

https://www.allanninal.dev/stripe/no-3ds-on-elevated-risk/

https://github.com/allanninal/stripe-fixes/tree/main/no-3ds-on-elevated-risk

---

63 **a connected account has no external account to pay out to**

A seller has been taking payments for five months. Their Stripe balance is a five-figure number and it has only ever gone up. There are no failed payouts to investigate, no errors in any log, and no alert anywhere, for the simple reason that nothing has ever been attempted: the account has no bank account attached, so automatic payouts have nowhere to send the money.

`stripe_missing_external_account.py`

https://www.allanninal.dev/stripe/no-external-account-attached/

https://github.com/allanninal/stripe-fixes/tree/main/no-external-account-attached

---

64 **live mode has no webhook endpoint, so nothing is ever pushed**

The first real payment arrives and the application does nothing with it. No order row, no fulfilment email, no account provisioned. The charge is right there in the Dashboard, marked succeeded. It all worked perfectly in development, every single time, because stripe listen was doing the delivering and nobody noticed that it was the only thing that ever had.

`stripe_live_webhook_endpoints.py`

https://www.allanninal.dev/stripe/no-live-webhook-endpoints/

https://github.com/allanninal/stripe-fixes/tree/main/no-live-webhook-endpoints

---

65 **no tax registrations while invoicing many countries**

Stripe Tax is enabled. automatic_tax.status reads complete on every invoice. The tax line is zero on all of them, including the ones going to Germany and the UK, and the reason given is not_collecting. Nothing is failing. Stripe calculated the tax correctly, and the correct answer given the registrations on file is nothing.

`stripe_tax_registrations.py`

https://www.allanninal.dev/stripe/no-tax-registrations-while-selling-abroad/

https://github.com/allanninal/stripe-fixes/tree/main/no-tax-registrations-while-selling-abroad

---

66 **no v2 event destination exists, so thin events never arrive**

The feature you turned on documents an event with a v1. prefix in its name. Somebody added it to the webhook endpoint, saved, and nothing happened — no delivery, no failure, no entry in the endpoint's log. The event is real and it is being generated. It is being handed to a kind of destination your account does not have.

`stripe_v2_event_destinations.py`

https://www.allanninal.dev/stripe/no-v2-event-destinations/

https://github.com/allanninal/stripe-fixes/tree/main/no-v2-event-destinations

---

67 **a live webhook endpoint points at a dev tunnel or http**

It worked for exactly one afternoon. The endpoint was registered during a development session, against a tunnel that happened to be running at the time; the tunnel was closed at the end of the day and the hostname stopped resolving. Every delivery attempt since then has failed before it reached any code you wrote.

`stripe_webhook_url_check.py`

https://www.allanninal.dev/stripe/non-https-or-tunnel-webhook-url/

https://github.com/allanninal/stripe-fixes/tree/main/non-https-or-tunnel-webhook-url

---

68 **off-session charges die on authentication_required**

A renewal fails. The customer's card is fine, it has not expired, it has money on it, and it worked when they first signed up. You retry off-session and it fails identically, so you retry again on a schedule and it fails identically again. The decline_code is authentication_required, and no number of retries will ever change it.

`stripe_offsession_mandates.py`

https://www.allanninal.dev/stripe/off-session-authentication-required-declines/

https://github.com/allanninal/stripe-fixes/tree/main/off-session-authentication-required-declines

---

69 **accounts stall at details_submitted false after link expiry**

Your database has four hundred acct_ rows and two hundred and ninety of them have never done anything at all. Support has a handful of tickets that all say some version of the same thing: the Stripe page said something went wrong, or the emailed link did nothing when they clicked it. Nobody connected the tickets to the rows, because the rows do not look like failures. They look like people who changed their minds.

`stripe_onboarding_stalled.py`

https://www.allanninal.dev/stripe/onboarding-abandoned-details-not-submitted/

https://github.com/allanninal/stripe-fixes/tree/main/onboarding-abandoned-details-not-submitted

---

70 **open invoices are weeks past due_date and nobody chases**

Invoiced customers pay when they are asked twice. The integration asks once: Stripe emails the invoice on finalization and then waits, because that is what collection_method=send_invoice means. Nobody enabled the reminder emails, so an invoice that went out in April is still sitting at open in July, and the customer's subscription never noticed.

`stripe_overdue_invoices.py`

https://www.allanninal.dev/stripe/open-invoices-past-due-date/

https://github.com/allanninal/stripe-fixes/tree/main/open-invoices-past-due-date

---

71 **pending invoice items that never reach an invoice**

Somebody added a setup fee in March. It is still there, sitting in the account with invoice: null, waiting for the next invoice to sweep it up. That customer cancelled in April, so there will never be a next invoice, and the fee has been quietly not-billed for five months while showing up in nobody's report of anything.

`stripe_pending_invoice_items.py`

https://www.allanninal.dev/stripe/orphaned-pending-invoice-items/

https://github.com/allanninal/stripe-fixes/tree/main/orphaned-pending-invoice-items

---

72 **past_due subscriptions keep their access forever**

Renewals have been failing for months and nobody noticed, because the customers are still logged in and still using the product. The entitlement check in your app asks whether the subscription is canceled. It is not canceled. It is past_due, which the check has never heard of, so the answer is yes, let them in.

`stripe_past_due_subs.py`

https://www.allanninal.dev/stripe/past-due-subscriptions-accumulating/

https://github.com/allanninal/stripe-fixes/tree/main/past-due-subscriptions-accumulating

---

73 **pause_collection with no resumes_at silently bills nothing**

One customer had a bad month, support paused their billing for a while, and everybody moved on. That was in February. The subscription still reads active in every report you have, it still shows up in the subscriber count, and it has not produced a single invoice since.

`stripe_pause_collection.py`

https://www.allanninal.dev/stripe/pause-collection-left-on-indefinitely/

https://github.com/allanninal/stripe-fixes/tree/main/pause-collection-left-on-indefinitely

---

74 **paused subscriptions never resume and never invoice again**

Somebody set trials to pause instead of dunning when no card was attached, which was the right call. Nobody built the other half. Two years of trials that ended without a card are sitting in paused, generating nothing, appearing in no past-due report, and waiting for a resume that is not coming.

`stripe_paused_subscriptions.py`

https://www.allanninal.dev/stripe/paused-subscriptions-never-resumed/

https://github.com/allanninal/stripe-fixes/tree/main/paused-subscriptions-never-resumed

---

75 **PaymentIntents have a null customer, so payments are orphaned**

Someone in support asks a simple question: how many times has this person bought from us? There is no answer. Their payments are in Stripe, four of them, all succeeded, all with the customer column empty. Same card, same email in the billing details, four separate strangers as far as Stripe is concerned — and as far as Radar is concerned too.

`stripe_orphan_payments.py`

https://www.allanninal.dev/stripe/payment-intents-with-null-customer/

https://github.com/allanninal/stripe-fixes/tree/main/payment-intents-with-null-customer

---

76 **payment link hit its completed-session limit and went dead**

The campaign link worked. Fifty people bought through it and then the orders stopped, on a link nobody touched, in a week nobody deployed. The link is still active, still resolving, still in the email that goes out every morning. It reached the number of completed sessions it was created with and closed itself.

`stripe_payment_link_limits.py`

https://www.allanninal.dev/stripe/payment-link-completion-limit-reached/

https://github.com/allanninal/stripe-fixes/tree/main/payment-link-completion-limit-reached

---

77 **Payment Link ends on Stripe's page, so fulfilment never fires**

The link works. The money arrives. The customer sees a Stripe page thanking them, closes the tab, and waits for a licence key that no code was ever asked to send. Nothing failed anywhere: the flow simply ends on Stripe's side and never comes back to yours.

`stripe_payment_link_fulfilment.py`

https://www.allanninal.dev/stripe/payment-link-hosted-confirmation-no-fulfilment/

https://github.com/allanninal/stripe-fixes/tree/main/payment-link-hosted-confirmation-no-fulfilment

---

78 **a deactivated Payment Link is still linked from your site**

Somebody deactivated a Payment Link in the Dashboard six weeks ago, for a good reason. The URL is still in a landing page, a scheduled email and a PDF invoice template, and every customer who clicks it lands on a Stripe page explaining that the link is no longer active. Your server never hears about any of it.

`stripe_inactive_payment_links.py`

https://www.allanninal.dev/stripe/payment-link-inactive-still-published/

https://github.com/allanninal/stripe-fixes/tree/main/payment-link-inactive-still-published

---

79 **payouts cannot be tied back to their balance transactions**

A single deposit arrives in the bank and finance asks the only reasonable question: what is in it? The obvious answer, GET /v1/balance_transactions?payout=po_x, comes back with an empty list. Not an error, not a permissions problem — an empty list, on a payout that definitely happened.

`stripe_payout_reconciliation.py`

https://www.allanninal.dev/stripe/payout-reconciliation-unavailable/

https://github.com/allanninal/stripe-fixes/tree/main/payout-reconciliation-unavailable

---

80 **a payout schedule left on manual strands the balance**

A seller opens a ticket asking where four months of money went. Everything you check says the account is fine: payouts are enabled, no requirements are outstanding, a verified bank account is attached and set as the default. There are no failed payouts to investigate because there are no payouts at all, and the reason is a single string in a settings hash nobody has read since the platform was built.

`stripe_manual_payout_schedule.py`

https://www.allanninal.dev/stripe/payout-schedule-left-on-manual/

https://github.com/allanninal/stripe-fixes/tree/main/payout-schedule-left-on-manual

---

81 **payouts fail with account_closed and nobody is watching**

The ledger says the payout was paid. The recipient says no money arrived. Both are true: the payout reached paid, the bank rejected the credit four days later, Stripe moved it to failed and returned the funds to your balance. Nothing in your system recorded the second half of that story, because nothing was reading the object after it went green.

`stripe_failed_payouts.py`

https://www.allanninal.dev/stripe/payouts-failing-bank-rejection/

https://github.com/allanninal/stripe-fixes/tree/main/payouts-failing-bank-rejection

---

82 **a Person's currently_due blocks the whole account**

The seller filled in everything your onboarding form asked for. The account object shows a business name, an address, a tax id and a bank account, and charges_enabled is still false. The one thing in requirements.currently_due is a string that starts person_1Mq and ends .verification.document, and there is no field anywhere in your product that corresponds to it.

`stripe_person_requirements.py`

https://www.allanninal.dev/stripe/person-requirements-outstanding/

https://github.com/allanninal/stripe-fixes/tree/main/person-requirements-outstanding

---

83 **platform-paused payouts were never unpaused**

In March, risk paused a seller while a chargeback pattern was investigated. In April the investigation closed with nothing found, the ticket was marked resolved, and everyone moved on. It is now September. The seller has been taking payments the whole time, the money has been accumulating, and the pause is still on, because unpausing was a manual step in a dashboard and nothing anywhere remembers that it was owed.

`stripe_platform_paused.py`

https://www.allanninal.dev/stripe/platform-paused-payouts-left-on/

https://github.com/allanninal/stripe-fixes/tree/main/platform-paused-payouts-left-on

---

84 **prices left at tax_behavior unspecified break tax math**

Somebody turns on automatic tax and the first invoice refuses to take a line item. The price is fine, the product is fine, the amount is right, and Stripe will not compute tax on it, because tax_behavior is unspecified and it genuinely does not know whether the 20 EUR on that price already has VAT in it or not.

`stripe_price_tax_behavior.py`

https://www.allanninal.dev/stripe/prices-with-tax-behavior-unspecified/

https://github.com/allanninal/stripe-fixes/tree/main/prices-with-tax-behavior-unspecified

---

85 **radar blocks payments and nobody reads the block reasons**

Support keeps hearing the same sentence: my card works everywhere else. The charge shows as failed with a message that says nothing, the customer's bank has no record of the attempt at all, and nobody on your side can say why. The payment never left Stripe.

`stripe_radar_blocks.py`

https://www.allanninal.dev/stripe/radar-blocked-payments-ignored/

https://github.com/allanninal/stripe-fixes/tree/main/radar-blocked-payments-ignored

---

86 **radar is blocking a large share of your charge attempts**

Conversion stepped down on a specific day and stayed there. Support is hearing the same sentence from different customers: the card works everywhere else. It does work everywhere else — those payments were stopped before they were ever sent to an issuer, by a rule somebody wrote here.

`stripe_block_rate.py`

https://www.allanninal.dev/stripe/radar-blocked-rate-overblocking/

https://github.com/allanninal/stripe-fixes/tree/main/radar-blocked-rate-overblocking

---

87 **radar reviews sit open for days while funds stay at risk**

Somebody added a review rule, which was the right instinct. The queue it feeds has forty-one items in it, the oldest is from three weeks ago, and nobody has opened the page since the week it was set up. Every one of those payments was already taken, and the ones that were not have quietly stopped being collectable.

`stripe_radar_review_queue.py`

https://www.allanninal.dev/stripe/radar-reviews-open-stale/

https://github.com/allanninal/stripe-fixes/tree/main/radar-reviews-open-stale

---

88 **refunds sit failed or requires_action and nobody notices**

Support issued the refund, the ticket was closed, and the money left your Stripe balance. Weeks later the same customer opens a dispute for the same transaction, because it never arrived. You now pay the amount twice and the dispute fee on top, and the only record of what went wrong was a status change on an object nobody was watching.

`stripe_refund_health.py`

https://www.allanninal.dev/stripe/refunds-failed-or-stuck/

https://github.com/allanninal/stripe-fixes/tree/main/refunds-failed-or-stuck

---

89 **report runs past data_available_end return short data**

The month-end report succeeded. The totals are lower than the Dashboard by a few thousand, and re-running the identical report the next morning produces a larger number. Nothing errored either time, which is the part that makes this take a day to find: a report that is short does not look any different from a report that is complete.

`stripe_report_interval.py`

https://www.allanninal.dev/stripe/report-interval-past-data-available-end/

https://github.com/allanninal/stripe-fixes/tree/main/report-interval-past-data-available-end

---

90 **a report run fails after the 200 and the CSV never lands**

Finance says the nightly export has been missing for a week. Your job logs say it succeeded every night, and they are not lying: creating a report run returned 200 and the job exited zero. The run failed twenty seconds later, in Stripe, where nobody was looking.

`stripe_report_runs.py`

https://www.allanninal.dev/stripe/report-run-failed-silently/

https://github.com/allanninal/stripe-fixes/tree/main/report-run-failed-silently

---

91 **requirements.past_due has already disabled the payouts**

There is a monitor. It runs daily, it reads every connected account, and it alerts when requirements.currently_due is not empty. It has been green for a month. A seller's payouts stopped eleven days ago and the monitor never said a word about it, because the field that would have told it apart from routine housekeeping is a different array on the same object.

`stripe_requirements_past_due.py`

https://www.allanninal.dev/stripe/requirements-past-due-disables-account/

https://github.com/allanninal/stripe-fixes/tree/main/requirements-past-due-disables-account

---

92 **save_default_payment_method off orphans the card after payment**

The signup works end to end. The customer enters a card, the first invoice is paid, the subscription goes active, and everyone moves on. A month later the renewal fails and the subscription has nothing on it to charge. The card that paid the first invoice is not gone — it was simply never made the subscription's default, because a flag nobody set defaults to off.

`stripe_save_default_pm.py`

https://www.allanninal.dev/stripe/save-default-payment-method-off/

https://github.com/allanninal/stripe-fixes/tree/main/save-default-payment-method-off

---

93 **subscriptions frozen on requires_action 3DS authentication**

European signups convert worse than everyone else's and support has nothing to go on. The card is good, the customer entered it correctly, and the bank was ready to approve the payment as soon as somebody answered a question. Nobody asked them the question. The invoice is still open, waiting.

`stripe_sca_stuck_subs.py`

https://www.allanninal.dev/stripe/sca-authentication-stuck-subscriptions/

https://github.com/allanninal/stripe-fixes/tree/main/sca-authentication-stuck-subscriptions

---

94 **invoiced subscriptions with no days_until_due never age**

The finance team asks for an aged receivables report and the answer comes back empty, which everyone reads as good news. It is not: these invoices are not current, they are undateable. days_until_due was never set on the subscriptions that generate them, so Stripe writes each invoice with due_date null, and an invoice with no due date cannot be overdue, cannot trigger a reminder, and cannot age into any bucket at all.

`stripe_days_until_due.py`

https://www.allanninal.dev/stripe/send-invoice-without-days-until-due/

https://github.com/allanninal/stripe-fixes/tree/main/send-invoice-without-days-until-due

---

95 **SetupIntents use on_session but you bill off-session**

The card saves cleanly. It shows up on the customer, the last four digits are right, and if the customer comes back and pays with it themselves it works every time. The renewal that runs at three in the morning fails with authentication_required, and so does the one after that. The card is saved; it was never authorised for anyone but the customer to use.

`stripe_setup_intent_usage.py`

https://www.allanninal.dev/stripe/setup-intent-on-session-for-off-session/

https://github.com/allanninal/stripe-fixes/tree/main/setup-intent-on-session-for-off-session

---

96 **SetupIntents are created but never confirmed by the client**

Support keeps hearing the same thing: they added a card, the page said it worked, and the next invoice failed anyway. In the API there are four hundred SetupIntents sitting at requires_confirmation, created over three months, none of which ever produced a mandate or a PaymentMethod on a Customer.

`stripe_setup_intents_stuck.py`

https://www.allanninal.dev/stripe/setup-intents-never-confirmed/

https://github.com/allanninal/stripe-fixes/tree/main/setup-intents-never-confirmed

---

97 **sigma scheduled query runs time out and email nothing**

Finance asks where the Monday numbers went, and it turns out they have not arrived for six weeks. Nobody raised it earlier because the alert for this failure is an email that does not appear, and an email that does not appear looks exactly like a week where nothing happened.

`stripe_sigma_runs.py`

https://www.allanninal.dev/stripe/sigma-scheduled-query-failing/

https://github.com/allanninal/stripe-fixes/tree/main/sigma-scheduled-query-failing

---

98 **paymentIntents sit in requires_payment_method for weeks**

The Payments page shows a long tail of incomplete payments that never resolve into anything. Checkout starts and successful payments have drifted apart by a factor nobody can explain, and the gap grows every week. Most of those intents were never going to succeed: they were created before the customer had done anything, and nothing ever went back to them.

`stripe_stale_intents.py`

https://www.allanninal.dev/stripe/stale-requires-payment-method-intents/

https://github.com/allanninal/stripe-fixes/tree/main/stale-requires-payment-method-intents

---

99 **a second-currency balance bucket can never be paid out**

The Stripe balance and the bank statements have been out by the same figure for months. It is not a rounding error and it is not a timing difference: it is a few hundred euros, on an account that pays out in dollars, sitting in a bucket that no payout has ever touched or ever will.

`stripe_stranded_currency.py`

https://www.allanninal.dev/stripe/stranded-currency-balance/

https://github.com/allanninal/stripe-fixes/tree/main/stranded-currency-balance

---

100 **active subscriptions with nothing to charge on renewal**

The subscription says active. The customer has access, the MRR chart counts them, and the renewal date is in the calendar. Then the renewal date arrives and the invoice fails, and it fails again next month, and Stripe never retries any of it — because there is nothing to retry against.

`stripe_sub_payment_method.py`

https://www.allanninal.dev/stripe/subscription-without-payment-method/

https://github.com/allanninal/stripe-fixes/tree/main/subscription-without-payment-method

---

101 **incomplete subscriptions die silently after 23 hours**

Someone filled in the card form, saw a spinner, and closed the tab believing they had subscribed. Stripe has a subscription for them in incomplete. In under a day that record becomes incomplete_expired, the open invoice is voided, and there is nothing left to recover — no charge, no error, no ticket.

`stripe_incomplete_subs.py`

https://www.allanninal.dev/stripe/subscriptions-stuck-incomplete/

https://github.com/allanninal/stripe-fixes/tree/main/subscriptions-stuck-incomplete

---

102 **terminal readers sit offline and take no payments**

A shop's card takings were zero all weekend. There is nothing in the Dashboard to look at: no declines, no failed PaymentIntents, no errors. That is the tell. A reader that is offline does not fail payments, it never starts them, so the evidence of the outage is an absence of records rather than a list of them.

`stripe_terminal_readers.py`

https://www.allanninal.dev/stripe/terminal-readers-offline/

https://github.com/allanninal/stripe-fixes/tree/main/terminal-readers-offline

---

103 **live charges fail with testmode_decline from test cards**

A customer writes in to say their card was declined. You try it yourself with 4242 4242 4242 4242 and it works fine, so you tell them to call their bank. It is not their bank. The charge failed with testmode_decline, which is Stripe saying that something in the request belonged to test mode and the request did not — and the card that “works fine” is the reason you cannot see it.

`stripe_live_mode_check.py`

https://www.allanninal.dev/stripe/testmode-decline-in-live-mode/

https://github.com/allanninal/stripe-fixes/tree/main/testmode-decline-in-live-mode

---

104 **the transfers capability is inactive so every transfer 400s**

One seller's payouts never arrive. Their checkout works, the charges land on the platform, the account object says charges_enabled: true, and the seller's own balance stays at zero. Every attempt to move the money returns a 400 that names a capability nobody on the team has ever read a value from.

`stripe_transfers_capability.py`

https://www.allanninal.dev/stripe/transfers-capability-inactive/

https://github.com/allanninal/stripe-fixes/tree/main/transfers-capability-inactive

---

105 **trials ending in days with no card on file**

Card-free trials are good for signups and they build a queue. Everyone who started a trial on the first of the month reaches its end on the same day, and the ones without a card all fail together. What happens next is one Stripe setting most teams have never opened, and its default is the loudest of the three options and the quietest to you.

`stripe_trial_no_card.py`

https://www.allanninal.dev/stripe/trial-ends-without-payment-method/

https://github.com/allanninal/stripe-fixes/tree/main/trial-ends-without-payment-method

---

106 **PaymentMethods are created but never attached to a customer**

The first charge works. The customer comes back a month later, picks their saved card, and the request fails with a sentence about a PaymentMethod that was previously used without being attached to a Customer. The pm_ id is right there in your database, it looks fine, and it will never work again.

`stripe_orphaned_payment_methods.py`

https://www.allanninal.dev/stripe/unattached-payment-methods-orphaned/

https://github.com/allanninal/stripe-fixes/tree/main/unattached-payment-methods-orphaned

---

107 **refunds nobody issued with reason expired_uncaptured_charge**

Somebody in the monthly review asks why the refund rate went from 1.2% to 3.1%. Support has no extra tickets. Nobody remembers approving a wave of refunds, and the finance export shows money going back out that never came in. Every one of those refunds was written by Stripe, and no customer asked for a single one.

`stripe_expired_capture_refunds.py`

https://www.allanninal.dev/stripe/uncaptured-charge-expiry-refunds/

https://github.com/allanninal/stripe-fixes/tree/main/uncaptured-charge-expiry-refunds

---

108 **undelivered events are aging out of the 30-day window**

The handler is fixed. The endpoint is enabled again. Now someone has to replay three weeks of missed events, and a quiet arithmetic problem is waiting: Stripe keeps events for 30 days, the oldest ones are on day 26, and the backfill script has not been written yet.

`stripe_event_retention.py`

https://www.allanninal.dev/stripe/undelivered-events-nearing-retention/

https://github.com/allanninal/stripe-fixes/tree/main/undelivered-events-nearing-retention

---

109 **unpaid subscriptions keep access and stop billing entirely**

A handful of customers are still logged in and still using everything, and the last money any of them sent you was months ago. Their subscriptions are not canceled and they are not past_due either. They are unpaid, which is the state Stripe parks a subscription in when it has finished trying and been told not to cancel.

`stripe_unpaid_subscriptions.py`

https://www.allanninal.dev/stripe/unpaid-subscriptions-still-provisioned/

https://github.com/allanninal/stripe-fixes/tree/main/unpaid-subscriptions-still-provisioned

---

110 **event types are firing that no endpoint subscribes to**

A whole class of business event is invisible to the application, and there is no error anywhere to explain it. The events exist. They are in GET /v1/events with the right data on them. They were never delivered, because no endpoint asked for them, and an event nobody asked for is not a failure — it is a non-event.

`stripe_unsubscribed_events.py`

https://www.allanninal.dev/stripe/unsubscribed-event-types-firing/

https://github.com/allanninal/stripe-fixes/tree/main/unsubscribed-event-types-firing

---

111 **requirements.errors explains the rejected document**

A seller has uploaded the same photo of their passport four times over two weeks. Your onboarding UI has said verification pending every time, because that is the only string it knows. Stripe has been returning the specific reason since the first attempt: the scan is greyscale. Nobody has ever read the field it is in, so the seller keeps sending the file that cannot pass.

`stripe_verification_errors.py`

https://www.allanninal.dev/stripe/verification-errors-unread/

https://github.com/allanninal/stripe-fixes/tree/main/verification-errors-unread

---

112 **no payment method domain registered, so wallets never show**

Apple Pay renders perfectly on localhost. It renders in Stripe's own demo. It is enabled in the Dashboard, the browser is Safari, the device has a card in the wallet — and on checkout.example.com the button is simply not there. No console error, no failed request, nothing in the Stripe logs. The wallet was filtered out before it had a chance to render, because Stripe does not recognise the domain asking for it.

`stripe_wallet_domains.py`

https://www.allanninal.dev/stripe/wallet-domain-not-registered/

https://github.com/allanninal/stripe-fixes/tree/main/wallet-domain-not-registered

---

113 **a webhook endpoint sits disabled after days of retries**

Orders stopped being fulfilled on a Tuesday. Nothing changed in your code that week, nothing appears in your application logs, and Stripe still shows the payments as succeeded. Nothing is arriving at all — Stripe gave up on the endpoint and stopped trying.

`stripe_webhook_health.py`

https://www.allanninal.dev/stripe/webhook-endpoint-disabled/

https://github.com/allanninal/stripe-fixes/tree/main/webhook-endpoint-disabled

---

114 **an endpoint subscribes to every event and floods the handler**

The webhook route is slow, and it is slow at the worst possible time: the end of the month, when renewals run. It handles four event types. It is being sent everything Stripe generates, and the other ninety-odd types still cost a request, a signature verification, and a database round trip before the handler decides it does not care.

`stripe_wildcard_events.py`

https://www.allanninal.dev/stripe/wildcard-enabled-events/

https://github.com/allanninal/stripe-fixes/tree/main/wildcard-enabled-events

---